

Virtualizzazione Insane

Christian Candela

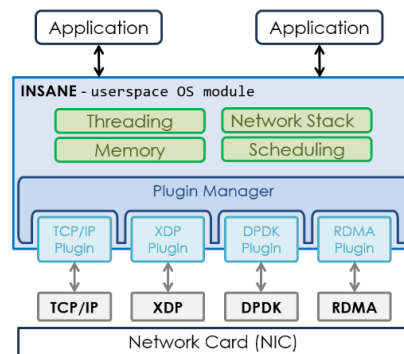
Indice

Capitolo 1.....	3
1.1 Cosa è INSANE	3
1.2 Obiettivo del progetto	3
1.3 Come funziona INSANE	4
Capitolo 2.....	6
2.1 Multi stage build	6
2.2 Single stage build.....	7
2.3 Avvio dei container.....	8
2.3.1 Flag privileged	8
2.3.2 Gestione della connettività.....	8
2.4 Volumi	11
Capitolo 3.....	12
3.1 Orchestratore.....	12
3.2 Gestione dei demoni	12
3.3 Primo avvio utilizzando k3s	13
3.3.1 Host Network.....	14
3.3.2 Flannel e Multus	14
3.4 Kubernetes	16
3.5 Creazione delle immagini per i pod.....	18
3.6 Registro delle immagini.....	18
3.6.1 Local registry k3s.....	19
3.6.2 Local registry Kubernetes v1	20
3.6.2 Local registry Kubernetes v2	21
3.7 Configmaps.....	21
3.8 File manifest	22
Capitolo 4.....	24
4.1 Test.....	24
4.2 Test completo Bare Metal.....	25
4.3 Test completo Docker	26
4.4 Test completo Kubernetes	27
4.5 Aggregazione dei dati	28
4.6 Rappresentazione dei dati	28

Capitolo 1

1.1 Cosa è INSANE

INSANE, acronimo di *Integrated aNd Selective Acceleration at the Network Edge*, è un middleware progettato per **accelerare la comunicazione delle applicazioni distribuite**, sia in ambienti cloud tradizionali sia in edge cloud, più vicini agli utenti. INSANE permette di sviluppare applicazioni dichiarando i requisiti di qualità di servizio per ogni canale di comunicazione. Il middleware si occuperà di **scegliere automaticamente il protocollo più adatto** per soddisfare tali requisiti. In questo modo, la logica di accelerazione di rete non è gestita dallo sviluppatore, ma dal middleware stesso.



1.2 Obiettivo del progetto

L'obiettivo del progetto è la realizzazione di una versione containerizzata di alcune applicazioni che usano INSANE e integrarle in un orchestratore. I **container** sono stati scelti perché rappresentano lo **standard per la distribuzione e l'esecuzione di applicazioni in cloud**. Essi includono tutte le dipendenze necessarie, facilitando il riutilizzo e l'orchestrazione.

Nel progetto sono stati containerizzati:

- **nsn-app**: applicazione demo che permette di selezionare due tipi di qualità del servizio, *reliable* o *non reliable*.

- **nsnd**: demone che gestisce la comunicazione tra applicazioni.

L'orchestratore utilizzato è **Kubernetes**, che gestisce il deployment e il ciclo di vita dei container.

1.3 Come funziona INSANE

La comunicazione tra l'applicazione e il demone avviene attraverso due canali: una **socket** e un'**area di memoria condivisa**. All'avvio il demone costruisce una socket Unix di tipo datagramma (*/tmp/nsnd_control_plane.socket*) che funge da pannello di controllo. Gli applicativi, come *nsn_app_perf*, la utilizzano per inviare messaggi di controllo come CONNECT o CREATE_STREAM al demone.

Il trasferimento dei dati tra il demone e l'applicazione è invece realizzato tramite la condivisione di uno spazio di memoria allocato all'interno delle **HugePages**. Una pagina standard è tipicamente di 4 KB mentre una HugePage è di 2 MB. L'utilizzo di queste pagine più grandi permette di ridurre l'overhead del kernel perché facilita la conversione tra indirizzo virtuale e fisico visto che il numero di entry nella tabella di conversione è minore. Inoltre, utilizzando pagine grandi anche la comunicazione attraverso interfacce di rete ne beneficia.

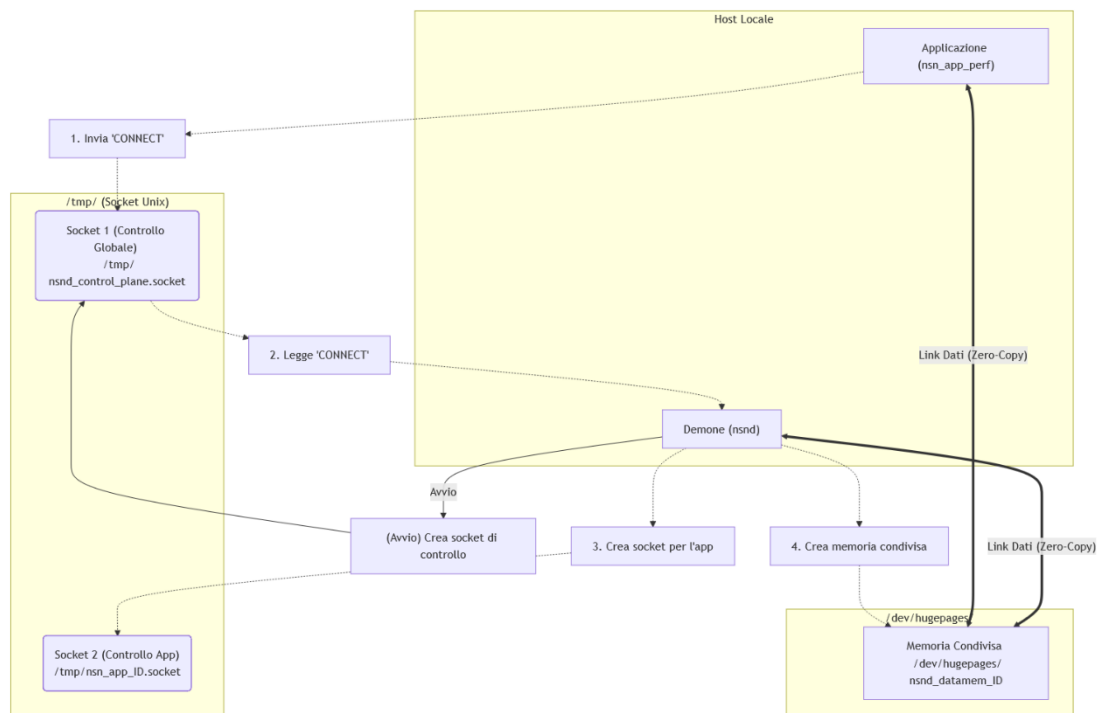
L'uso della memoria condivisa permette di realizzare un **modello zero-copy** in cui l'applicazione e il demone mappano lo stesso file di memoria. In questo modo, quando l'app scrive dei pacchetti, il demone vede immediatamente gli stessi byte senza bisogno di copiarli tra i processi. Quando un'applicazione si connette al demone vengono quindi creati due file */dev/hugepages/nsnd_datamem_<id>* e */tmp/nsn_app<id>*. Il primo rappresenta l'area di memoria condivisa utilizzata dal demone e dell'applicazione per scambiarsi dati. Il secondo è una socket Unix usata dal demone per mandare messaggi di controllo all'applicazione. L'id permette di identificare una specifica applicazione.

Per la comunicazione con il mondo esterno il demone utilizza **plugin** selezionati in base alla qualità del servizio richiesta, separando così il core dalla logica di I/O. Nel

caso dell'app di test la scelta viene fatta su una comunicazione *reliable* che utilizza il plugin TCP e una *non reliable* che utilizza invece il plugin UDP.

In sintesi, la comunicazione con il middleware avviene così: l'app chiede via **socket** al demone di connettersi. Il demone crea la **memoria condivisa** nelle HugePages a cui la app avrà accesso. A questo punto l'applicazione e il demone comunicano con la memoria condivisa senza passare per la socket.

Prima dell'avvio del demone è necessario definire alcuni parametri di configurazione nel file **nsnd.cfg** come, ad esempio, l'ip del nodo locale e l'ip del nodo con il quale si vuole comunicare.



Capitolo 2

Per la containerizzazione delle applicazioni è stato necessario realizzare due **Dockerfile**: uno per il demone e uno per l'app di test. Durante la stesura di questi file ho seguito due strade differenti: **multi stage build** e **single stage build**.

2.1 Multi stage build

L'idea del multi stage build è quella di usare **un'immagine temporanea** chiamata **builder** in cui installare tutti gli strumenti necessari alla compilazione come ad esempio **gcc**. Copiare poi il codice da compilare e usare lo script *build.sh* per effettuare la compilazione. Qui sotto è possibile vedere la creazione dell'immagine builder:

```
FROM ubuntu:latest AS builder
RUN apt-get update && apt-get install -y \
    build-essential \
    gcc \
    make \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . .

RUN mkdir -p build
RUN chmod +x ./build.sh && ./build.sh release
```

Nel secondo stage si parte da un'immagine pulita di **Ubuntu** in cui si copiano solo i file necessari ottenuti dal build precedente. Quest'immagine sarà quella che poi verrà utilizzata nel container.

```
FROM ubuntu:latest
WORKDIR /app

COPY --from=builder /app/build/nsn-perf /app/
COPY --from=builder /app/nsn-app.cfg /app/
COPY --from=builder /app/datapaths/libudpsoc.so /app/datapaths/

CMD ["/app/nsn-perf", "source", "-n", "10", "-s", "1024"]
```

2.2 Single stage build

Nel **single stage** build vengono copiati soltanto i file necessari all'esecuzione delle applicazioni. Nel caso del demone vengono copiati gli eseguibili, il file di configurazione e i plugin per la comunicazione con l'esterno. Inoltre, viene direttamente avviato il demone attraverso **CMD** ["/app/nsnd"]. Di seguito il file *Dockerfile.daemon*:

```
FROM ubuntu:latest

RUN mkdir -p /app/datapaths
COPY ./build/nsnd /app/
COPY ./build/nsnd.cfg /app/
COPY ./build/libnsn.o /app/
COPY ./datapaths/udpsock.o /app/datapaths
COPY ./datapaths/libudpsock.so /app/datapaths
COPY ./datapaths/libtcpsock.so /app/datapaths

WORKDIR /app/

CMD ["/app/nsnd"]
```

Per creare l'immagine dell'**applicazione di test** invece vengono copiati solo l'eseguibile e il file di configurazione. Inoltre, non viene eseguito nessun comando all'avvio con CMD in modo che sia possibile manualmente eseguire tutti i comandi di test necessari. Di seguito il file *Dockerfile.app*:

```
FROM ubuntu:latest

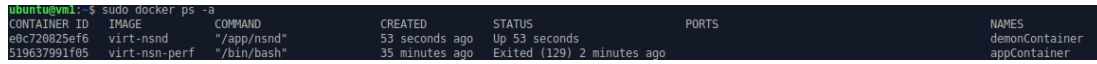
COPY ./build/nsn-perf /app/
COPY ./build/nsn-app.cfg /app/

WORKDIR /app/
```

In sintesi, il multi stage build consente di ottenere immagini più leggere e prive di strumenti di compilazione, mentre il single stage è più semplice da gestire quando i binari sono già disponibili. Per questo progetto ho **scelto di utilizzare il single stage**

build, in quanto garantiva una gestione più semplice e diretta delle immagini, dato che i binari erano già disponibili.

2.3 Avvio dei container



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e8c720825ef6	virt-nsnd	"/app/nsnd"	53 seconds ago	Up 53 seconds		demonContainer
b19637991f05	virt-nsn-perf	"/bin/bash"	35 minutes ago	Exited (129) 2 minutes ago		appContainer

Nell'avvio dei container in **Docker** ho riscontrato due problemi: l'accesso di Docker alla cartella `/dev/` necessaria per accedere alle **HugePages** e la connettività tra i due demoni che eseguivano su due macchine virtuali separate.

2.3.1 Flag privileged

Per permettere l'accesso alle HugePages nel comando `docker run` ho aggiunto il flag `--privileged` che serve a dare al container quasi tutti i **privilegi dell'host** rimuovendo molte delle limitazioni di sicurezza normalmente applicate da Docker. Visto che nel container è necessario montare `/dev/hugepages` il flag è necessario.

2.3.2 Gestione della connettività

2.3.2.1 Network host

Per la gestione della **connettività** tra i due demoni una prima soluzione che ho sperimentato è stata usare il flag `--network host`. In questo modo il container non utilizza una rete isolata ma **condivide lo stack di rete dell'host**. I processi dentro al container vedono le interfacce di rete dell'host come se fossero le proprie. L'avvio del demone è stato effettuato con il seguente comando:

```
docker run --privileged --network host -it \  
  --name demonContainer \  
  -v /tmp:/tmp \  
  -v /dev/hugepages:/dev/hugepages \  
  virt-nsn-perf2
```

mentre per l'applicazione di test è stato usato questo:

```
docker run --privileged -it \  
  --name appContainer \  
  -v /tmp:/tmp \  
  -v /dev/hugepages:/dev/hugepages \  
  virt-nsn-perf
```

In questo caso nel file di configurazione del demone gli indirizzi ip assegnati corrispondevano a quelli delle macchine virtuali che ospitavano i container. Di seguito un estratto del file:

```
#####  
# Peers' network configuration  
[peers.peer0]  
udpsock_ip="192.168.123.57"  
tcpsock_ip="192.168.123.57"  
udpdtpdk_ip="192.168.123.57"  
  
#####  
# Local PLUGINS config  
[plugins.udpsock]  
ip="192.168.123.147"  
  
[plugins.tcpsock]  
ip="192.168.123.147"
```

Sebbene questa soluzione abbia garantito il **corretto funzionamento** della comunicazione, comporta una **perdita totale dell'isolamento** di rete dei container. Tutti i processi hanno infatti accesso diretto alla rete dell'host, generando problemi di sicurezza e di portabilità.

2.3.2.2 Macvlan

La seconda soluzione che ho utilizzato è stata costruire una rete macvlan in modo da migliorare l'isolamento dei container. Una rete **macvlan** è un tipo di rete Docker che permette a un container di avere **un indirizzo ip sulla stessa rete fisica dell'host**. Dal punto di vista della rete esterna, il container si comporta come una **macchina fisica separata** con il proprio indirizzo MAC e il proprio ip.

Per eseguire INSANE ho creato la rete *macvlan* su entrambe le VM, in modo da permettere ai container di comunicare tra loro usando indirizzi ip appartenenti alla stessa sottorete.

Su VM1:

```
docker network create -d macvlan \
  --subnet=192.168.123.0/24 \
  --gateway=192.168.123.1 \
  -o parent=enp18s0 \
  macvlan
```

Su VM2:

```
docker network create -d macvlan \
  --subnet=192.168.123.0/24 \
  --gateway=192.168.123.1 \
  -o parent=enp18s0 \
  macvlan
```

A questo punto nel comando di avvio del container che esegue il demone è stato eliminato il flag *--network host* ed è stato aggiunto il flag che indica **l'ip che il container avrà nella rete macvlan**. Di seguito il comando per eseguire il demone:

```
docker run --privileged -it \
  --name demonContainer \
  --network macvlan \
  --ip 192.168.123.211 \
  -v /tmp:/tmp \
  -v /dev/hugepages:/dev/hugepages \
  virt-nsnd
```

Questo il comando per eseguire l'applicazione di test:

```
docker run --privileged -it \
  --name demonContainer \
  --network macvlan \
  --ip 192.168.123.212 \
  -v /tmp:/tmp \
```

```
-v /dev/hugepages:/dev/hugepages \
virt-nsnd
```

Ovviamente nel file di configurazione del demone sono stati inseriti gli ip assegnati nella rete macvlan.

```
#####
# Peers' network configuration
[peers.peer0]
udpsock_ip="192.168.123.212"
tcpsock_ip="192.168.123.212"
udpdtpdk_ip="192.168.123.212"

#####
# Local PLUGINS config
[plugins.udpsock]
ip="192.168.123.211"

[plugins.tcpsock]
ip="192.168.123.211"
```

Questa configurazione ha permesso ai container di comunicare correttamente tra loro mantenendo un livello di isolamento superiore rispetto a *--network host*. Tuttavia, l'uso di **macvlan** richiede un'attenta configurazione della rete fisica sottostante.

2.4 Volumi

Ultimo appunto va fatto sui **volumi a cui può accedere il container** che in questo caso sono due. Il volume */tmp* è montato nel container nella posizione */tmp* ed è necessario per mandare messaggi di controllo tra demone e applicazione. Il volume */dev/hugepages* è montato nel container nella posizione */dev/hugepages* necessario per lo scambio di dati tra container e applicazione di test.

Capitolo 3

3.1 Orchestratore

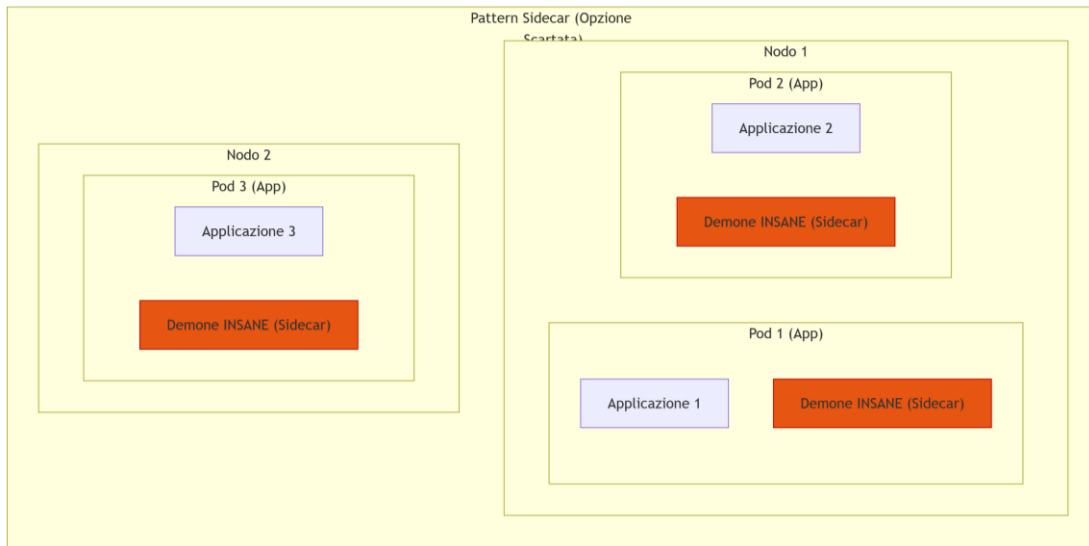
Per gestire in maniera più efficiente i container del progetto è stato utilizzato un **orchestratore**, nello specifico *Kubernetes*. Un orchestratore consente di automatizzare diverse operazioni di gestione, come il deployment, il monitoraggio e il riavvio dei container in caso di malfunzionamenti.

Grazie a questa soluzione, dalla macchina virtuale principale (VM1) è stato possibile controllare non solo i container locali, ma anche quelli in esecuzione su VM2. L'affidabilità complessiva del sistema risulta aumentata, poiché in caso di crash di un container questo viene automaticamente sostituito da una nuova istanza.

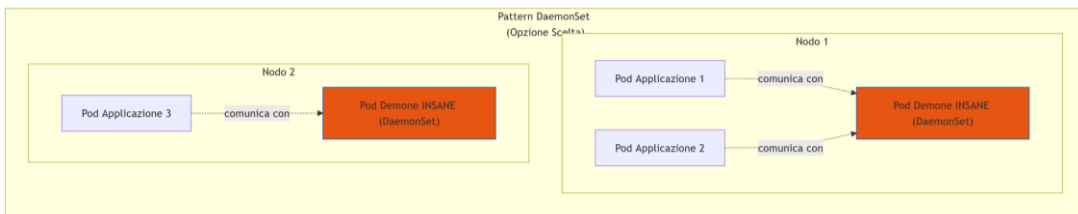
Un ulteriore vantaggio riguarda la gestione della rete: **Kubernetes** permette infatti di definire reti interne al cluster, non esposte all'esterno, che collegano direttamente i vari pod. Ciò semplifica la comunicazione tra i componenti del sistema e garantisce un migliore livello di isolamento.

3.2 Gestione dei demoni

Per la realizzazione del cluster che avrebbe ospitato INSANE è stato necessario scegliere come **gestire i demoni** al suo interno. Le due opzioni possibili erano il pattern *Sidecar* oppure *DaemonSet*. Il pattern *Sidecar* prevede che un **container ausiliario** venga eseguito nello stesso Pod dell'applicazione principale, condividendo con essa rete e risorse locali.



Un *DaemonSet*, invece, garantisce che un Pod specifico venga eseguito su tutti i nodi del cluster, assicurando così la presenza costante di un demone a livello di nodo.



Poiché INSANE è stato progettato come servizio di accelerazione di rete che opera a livello di nodo, l'approccio con *DaemonSet* risulta più corretta dal mio punto di vista: in questo modo ogni nodo dispone di un'istanza del demone, accessibile dalle applicazioni containerizzate. L'alternativa con *Sidecar* sarebbe comunque possibile, ma introdurrebbe duplicazione e maggiore overhead. Inoltre, il demone necessita di un file di configurazione in cui devono essere specificati l'indirizzo ip locale e quello del destinatario: con il modello *Sidecar*, ogni Pod avrebbe sia l'applicazione di test sia il demone, ciascuno con un file di configurazione dedicato, che dovrebbe contenere l'ip assegnato al Pod e quello del Pod con cui vuole comunicare.

3.3 Primo avvio utilizzando k3s

La prima soluzione sperimentata è stata quella di far eseguire i pod su **k3s** cioè una versione leggera e compatta di Kubernetes. Ho scelto questa distribuzione perché l'avevo già testata durante il corso di **Sistemi Distribuiti M**. Inoltre, mi era stato

consigliato di non eseguire distribuzioni troppo pesanti visto le risorse limitate delle due macchine virtuali. Questa **scelta però si è rivelata non efficace** per raggiungere l'obiettivo desiderato, anche questa volta infatti mi sono trovato a dover risolvere problemi di connettività tra i demoni. Questa fase però si è rivelata utile e mi ha permesso di capire meglio come scrivere i vari **manifest yaml** necessari per il deployment dei pod. Andrò adesso ad elencare le varie soluzioni che ho provato ad adottare per riuscire a realizzare la comunicazione tra i demoni. La trattazione completa del contenuto dei file yaml sarà invece svolta in una sezione successiva.

3.3.1 Host Network

La prima soluzione adottata per far comunicare i due pod consisteva nell'utilizzare **l'interfaccia di rete dell'host**, come avevo fatto in precedenza per i container. In questo caso nel file yaml è presente la riga *hostNetwork: True* che associa al pod l'interfaccia di rete dell'host. In questo caso quindi il file di configurazione dei demoni aveva come ip locale l'ip della macchina virtuale su cui stava eseguendo e l'ip di destinazione quello dell'altra macchina virtuale. Questa soluzione ovviamente non era adatta allo scopo visto che **non isolava per nulla i pod** dalla macchina host. Era necessario utilizzare un *CNI* come *Flannel* o *Calico* cioè un *Container Network Interface* in modo da poter gestire il network dei container

3.3.2 Flannel e Multus

La prima scelta per un **CNI** è ricaduta su **Flannel** in quanto già presente in K3s. Insieme a questo CNI avevo intenzione di utilizzare **Multus** cioè un meta-plugin di rete che a differenza dei CNI classici permette di assegnare più interfacce di rete contemporaneamente ai pod. Ho quindi proceduto ad installare Multus in modo da poter creare una rete **macvlan** e assegnare ip statici ai pod.

Dopo aver installato Multus ho creato due **NetworkAttachmentDefinition (NAD)**, **cioè**, due manifest yaml in cui specificare i parametri di rete per VM1:

```

apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-vm1
  namespace: default
spec:
  config: '{
    "cniVersion": "0.3.1",
    "type": "macvlan",
    "master": "enp18s0",
    "mode": "bridge",
    "ipam": {
      "type": "static",
      "addresses": [
        {
          "address": "192.168.123.211/24",
          "gateway": "192.168.123.1"
        }
      ]
    }
  }'
```

E per VM2:

```

apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-vm2
  namespace: default
spec:
  config: '{
    "cniVersion": "0.3.1",
    "type": "macvlan",
    "master": "enp18s0",
    "mode": "bridge",
    "ipam": {
      "type": "static",
      "addresses": [
        {
          "address": "192.168.123.212/24",
          "gateway": "192.168.123.1"
        }
      ]
    }
  }'
```

A questo punto nel manifest dei pod ho eliminato *hostNetwork: True* ed inserito *k8s.v1.cni.cncf.io/networks: macvlan-vm1* per VM1 e *k8s.v1.cni.cncf.io/networks: macvlan-vm2* per VM2. Nonostante fosse tutto corretto però i due pod non comunicavano. Andando ad osservare i pod in esecuzione Multus eseguiva correttamente su entrambi i nodi (*kube-multus-ds-7sl5h 1/1 Running*) ma eseguendo il comando *ip a* nei pod l'interfaccia di rete non compariva, come se il NAD non venisse considerato. Ho sospettato ci fosse qualche **problema con il CNI** principale cioè Flannel visto che non c'era nessun pod in esecuzione legato ad esso. Ho quindi provato ad installarlo da zero non ottenendo però nessun risultato. Ho allora deciso di cambiare CNI e ad **installare Calico** e di utilizzarlo come CNI principale senza Multus. Il problema in questo caso era che i pod di calico (*calico-node*) erano in stato running ma con 0/1 ready per cui Calico non funzionava correttamente. Uno dei problemi forse era il **CIDR (Classless Inter-Domain Routing)** cioè l'intervallo di ip associato alla rete Calico. Io cercavo di utilizzare la rete su *192.168.0.0/16* ma il file *calico.yaml* impostava la propria rete su *10.244.0.0/16*. Anche la modifica di questo file però non ha risolto il problema. A questo punto ho chiesto aiuto a Lorenzo il quale mi ha consigliato di provare ad **utilizzare Kubernetes** e non la distribuzione K3s.

3.4 Kubernetes

Per proseguire nella realizzazione del progetto ho quindi installato direttamente **kubeadm, kubelet e kubectl** insieme a **containerd**. La realizzazione del cluster però è stata abbastanza difficoltosa. Il primo problema che ho riscontrato riguardava la rete dei nodi che comunicavano utilizzando l'interfaccia di rete *192.168.122.0/24* che però era destinata alla comunicazione con l'esterno e non ad effettuare i test per INSANE. Per risolvere il problema sul worker ho modificato il file */var/lib/kubelet/kubeadm-flags.env* e aggiunto il flag *-ip=192.168.123.57* in modo da utilizzare la rete corretta. Per quanto invece riguarda il master ho modificato il comando di inizializzazione del cluster inserendo il flag *-apiserver-advertise-address=192.168.123.147*.

Dopo aver installato kubeadm, kubelet e kubectl era necessario avere un **CNI** in modo che fosse possibile la comunicazione tra i pod. Come prima opzione ho deciso di optare nuovamente per **Flannel** ma anche in questo caso **ho riscontrato diverse problematiche** nell'installazione. Nuovamente i pod necessari all'esecuzione del CNI

non si avviavano. Inoltre, utilizzando Flannel non avrei potuto impostare un ip statico ad uno specifico pod per questo ho deciso di accantonare Flannel e provare con **Calico**.

Per l'installazione di Calico ho provato ad utilizzare **Tigera Operator** che è un operatore che semplifica l'installazione, la configurazione e la gestione di Calico. In questa fase credo però di aver fatto confusione con la configurazione dei **CIDR** non riuscendo a far partire correttamente il CNI che continuava a non connettere i vari pod. Alla fine, ho deciso di provare nuovamente ad **installare manualmente Calico** senza utilizzare ulteriori tool e a settargli gli ip dei pod nel pool *10.244.0.0/24*. Per scoprire se la rete funzionasse davvero ho creato due pod ed eseguito su uno di essi il ping verso l'ip dell'altro e ottenuto un risultato positivo ho potuto proseguire. A questo punto **potevo definire un ip statico ai pod** dei demoni permettendo loro di comunicare. Nel manifest yaml del demone che avrebbe eseguito sulla VM1 ho quindi inserito la stringa *cni.projectcalico.org/ipAddrs: "[\"10.244.0.100\"]"* mentre per il demone destinato alla VM2 la stringa *cni.projectcalico.org/ipAddrs: "[\"10.244.0.200\"]"*. In questo modo i due pod eseguiranno su quegli specifici ip che saranno inseriti nel file di configurazione dei demoni.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: nsnd-daemon-vm1
spec:
  selector:
    matchLabels:
      app: nsnd-daemon-vm1
  template:
    metadata:
      labels:
        app: nsnd-daemon-vm1
      annotations:
        cni.projectcalico.org/ipAddrs: "[\"10.244.0.100\"]"
```

```
ubuntu@vm1:~$ kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE   NOMINATED NODE   READINESS GATES
nsn-perf-pod        1/1     Running   435 (59s ago)  33d   10.244.0.65   vm1    <none>           <none>
nsnd-daemon-vm1-7mxtn 1/1     Running   1 (2m17s ago)  4m10s 10.244.0.100  vm1    <none>           <none>
nsnd-daemon-vm2-7wgcf 1/1     Running   0           2m53s 10.244.0.200  vm2    <none>           <none>
```

3.5 Creazione delle immagini per i pod

Per avviare i pod è necessario fornire loro **un'immagine da eseguire all'interno del container**. La costruzione delle immagini è stata affrontata all'inizio della relazione; in questa fase è necessario effettuare solo una piccola modifica al Dockerfile del demone per poter utilizzare poi l'immagine nel cluster Kubernetes. La modifica consiste nel cancellare la riga che copia il file di configurazione. Il *Dockerfile.daemonpod* appare quindi così:

```
FROM ubuntu:latest

RUN mkdir -p /app/datapaths
COPY ./build/nsnd /app/
#COPY ./build/nsnd.cfg /app/
COPY ./build/libnsn.o /app/
COPY ./datapaths/udpsock.o /app/datapaths
COPY ./datapaths/libudpsock.so /app/datapaths
COPY ./datapaths/libtcpsock.so /app/datapaths

WORKDIR /app/

CMD ["/app/nsnd"]
```

La gestione della configurazione sarà affidata alle **configmap** che saranno trattate in una sezione successiva.

Il Dockerfile **dell'applicazione di test invece non richiede nessuna modifica** perché l'immagine generata per eseguire su Docker può essere utilizzata direttamente su Kubernetes.

3.6 Registro delle immagini

Per permettere ai pod di scaricare le immagini è stato necessario creare un **server locale** nel quale fosse possibile eseguire il pull e il push. La realizzazione di questo server è stato un altro punto dolente soprattutto perché prima Docker e poi Containerd effettuavano **richieste https e ricevevano risposte http** considerate non sicure e che

quindi non permettevano al pod di scaricare l'immagine. Comincerò prima ad illustrare la creazione del server utilizzato per i test con K3s per poi passare alle due soluzioni tentate per eseguire i pod su Kubernetes.

3.6.1 Local registry k3s

Per realizzare il registro locale su cui avrei caricato le immagini ho semplicemente avviato un **container sulla porta 5000** raggiungibile da tutti i nodi della rete collegandosi all'ip della VM1 (*docker run -d -p 5000:5000 --restart:always --name registry registry:2*).

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
519637991f05	virt-nsn-perf	"/bin/bash"	34 minutes ago	Exited (129) 31 seconds ago		appContainer
8731455ee712	registry:2	"/entrypoint.sh /etc-"	50 minutes ago	Up 50 minutes	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	registry

Il problema è che ottenevo questo errore: *"he push refers to repository [192.168.123.147:5000/virt-nsn-perf]Get "https://192.168.123.147:5000/v2/": http: server gave HTTP response to HTTPS client"* perché appunto la **connessione non era considerata sicura**. Per ovviare al problema è stato necessario modificare i **mirror** nel file */etc/rancher/k3s/registries.yaml*.

```
mirrors:
  "localhost:5000":
    endpoint:
      - "http://localhost:5000"
```

E creare anche il file */etc/docker/daemon.json* in cui inserire la stringa

```
{
  "insecure-registries": ["192.168.123.147:5000"]
}
```

A questo punto nel manifest file è sufficiente specificare l'immagine del pod.

```
- name: nsn-perf-container
  image: 192.168.123.147:5000/virt-nsn-perf:latest
```

3.6.2 Local registry Kubernetes v1

Una volta disinstallato k3s e installato Kubernetes ho cercato di creare un **registro delle immagini che fosse interno al cluster**. Per fare ciò ho creato prima un manifest file per costruire il pod che avrebbe contenuto al suo interno il registry.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: local-registry
  namespace: kube-system
spec:
  replicas: 1
  selector:
    matchLabels:
      app: local-registry
  template:
    metadata:
      labels:
        app: local-registry
    spec:
      containers:
        - name: registry
          image: registry:2
          ports:
            - containerPort: 5000
```

Poi un **service** che permetteva agli altri pod del cluster di accedere al registry locale usando *local-registry.kube-system.svc.cluster.local:5000*

```
apiVersion: v1
kind: Service
metadata:
  name: local-registry
  namespace: kube-system
spec:
  type: ClusterIP
  ports:
    - port: 5000
      targetPort: 5000
  selector:
    app: local-registry
```

Riscontravo nuovamente però il problema delle **richieste http non sicure**. Il pod provava a scaricare l'immagine ma otteneva un errore. A questo punto ho cercato delle soluzioni per comunicare al cluster che quel registro fosse sicuro senza però riuscirci. Ho quindi deciso di utilizzare nuovamente il registro locale della VM1.

3.6.2 Local registry Kubernetes v2

Ho creato nuovamente un container contenente l'immagine registry ed accessibile alla porta 5000. Dovevo modificare nuovamente le impostazioni di sicurezza per permettere di **gestire le risposte http** provenienti dal registro. In questo caso è stato necessario modificare il file `/etc/containerd/config.toml` aggiungendo questa riga in modo da permettere a containerd di considerare il **registro locale come sicuro**.

```
[plugins."io.containerd.grpc.v1.cri".registry.mirror."192.168.123.147:5000"]
    endpoint = ["http://192.168.123.147:5000"]
```

A questo punto quindi potevo inserire nuovamente *"image: 192.168.123.147:5000/virt-nsn-perf:latest"* nel manifest per far scaricare l'immagine dal registro locale.

3.7 Configmaps

Tramite le **ConfigMaps** è possibile gestire i file di configurazione separatamente dai container. Possiamo quindi fornire dati di configurazione ai pod senza dover ricostruire le immagini del container.

Nel caso del progetto ho creato due configmap, una per ciascun demone.

```
kubectl create configmap nsnd-config-node1 --from-file=nsnd.cfg=./configs/nsnd-node1.cfg
kubectl create configmap nsnd-config-node2 --from-file=nsnd.cfg=./configs/nsnd-node2.cfg
```

I file *nsnd-node1.cfg* e *nsnd-node2.cfg* contengono il file di configurazione dei demoni con gli ip definiti nel manifest di creazione dei pod. Nel manifest del pod, la **ConfigMap** viene montata come **volume** in questo modo:

```
- name: config-volume
  configMap:
    name: nsnd-config-node1
```

3.8 File manifest

Un ultimo aspetto da gestire nel manifest dei pod riguarda le **risorse e i volumi** a cui il pod può accedere.

Come descritto nel primo capitolo per utilizzare INSANE è necessario che il pod abbia accesso a */tmp* e */dev/hugepages*. Nel manifest sono quindi stati specificati questi due volumi (insieme alla configmap).

```
volumes:
- name: tmp-volume
  hostPath:
    path: /tmp
    type: Directory
- name: hugepages-volume
  hostPath:
    path: /dev/hugepages
    type: Directory
- name: config-volume
  configMap:
    name: nsnd-config-node1
```

Per utilizzare le HugePages è necessario aggiungere all'interno del manifest nel quale si specifica la **quantità di risorse necessarie** al pod per funzionare.

```
resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
    hugepages-2Mi: "2Gi"
  limits:
    cpu: "2"
    memory: "8Gi"
```

```
hugepages-2Mi: "2Gi"
```

In questo caso il container richiede un 1Gb di memoria ram e **2Gb di HugePages da 2Mb**. Inoltre, La sezione *limits* definisce il massimo delle risorse che il container può consumare.

Capitolo 4

4.1 Test

Per quanto riguarda la parte di test ho scelto di valutare il **tempo di invio dei messaggi** da un'applicazione all'altra. I test sono stati eseguiti in tre scenari: **bare metal**, **Docker** e **Kubernetes**. Per ogni ambiente sono stati inviati i messaggi sia con protocollo UDP che con protocollo TCP variando anche la grandezza del payload in bytes. Le due dimensioni scelte per i test sono state 512 bytes e 1024 bytes. Per ogni combinazione di ambiente, protocollo e grandezza del messaggio sono stati eseguiti cinque test consecutivi ed è stato successivamente calcolato il tempo medio.

4.2 Test completo Bare Metal

In questo caso il test è stato eseguito avendo per ognuna delle due **macchina virtuali** il demone e l'app di test in esecuzione.

UDP/Metal			TCP/Metal		
Tempo (ms)	Payload	Numero	Tempo	Payload	Numero
46.075	512	10000	53.397	512	10000
46.177	512	10000	46.665	512	10000
47.339	512	10000	47.720	512	10000
46.259	512	10000	46.506	512	10000
48.211	512	10000	46.300	512	10000
87.127	1024	10000	87.998	1024	10000
87.425	1024	10000	89.275	1024	10000
87.262	1024	10000	87.862	1024	10000
86.201	1024	10000	88.023	1024	10000
86.322	1024	10000	87.899	1024	10000
446.752	512	100000	508.954	512	100000
447.394	512	100000	483.054	512	100000
447.303	512	100000	499.776	512	100000
446.958	512	100000	480.449	512	100000
447.012	512	100000	483.102	512	100000
875.058	1024	100000	880.512	1024	100000
874.201	1024	100000	880.800	1024	100000
875.123	1024	100000	881.101	1024	100000
874.798	1024	100000	880.769	1024	100000
874.223	1024	100000	881.223	1024	100000

4.3 Test completo Docker

In questo caso il test è stato eseguito avviando per ogni macchina virtuale due **container**. Uno eseguiva il demone e l'altro l'applicazione di test.

UDP/Docker			TCP/Docker		
Tempo (ms)	Payload	Numero	Tempo (ms)	Payload	Numero
46.596	512	10000	88.524	512	10000
57.734	512	10000	90.041	512	10000
59.434	512	10000	91.537	512	10000
46.499	512	10000	89.077	512	10000
46.389	512	10000	89.051	512	10000
87.337	1024	10000	94.299	1024	10000
87.365	1024	10000	94.121	1024	10000
87.371	1024	10000	95.201	1024	10000
87.361	1024	10000	94.325	1024	10000
86.788	1024	10000	93.982	1024	10000
445.726	512	100000	465.726	512	100000
448.526	512	100000	468.526	512	100000
447.231	512	100000	470.234	512	100000
448.432	512	100000	468.286	512	100000
447.873	512	100000	568.328	512	100000
875.109	1024	100000	881.354	1024	100000
876.012	1024	100000	881.524	1024	100000
875.546	1024	100000	881.021	1024	100000
875.321	1024	100000	881.875	1024	100000
876.231	1024	100000	881.535	1024	100000

4.4 Test completo Kubernetes

In questo caso il test è stato avviando due **pod** per ogni macchina virtuale. Uno eseguiva il demone e l'altro l'applicazione di test.

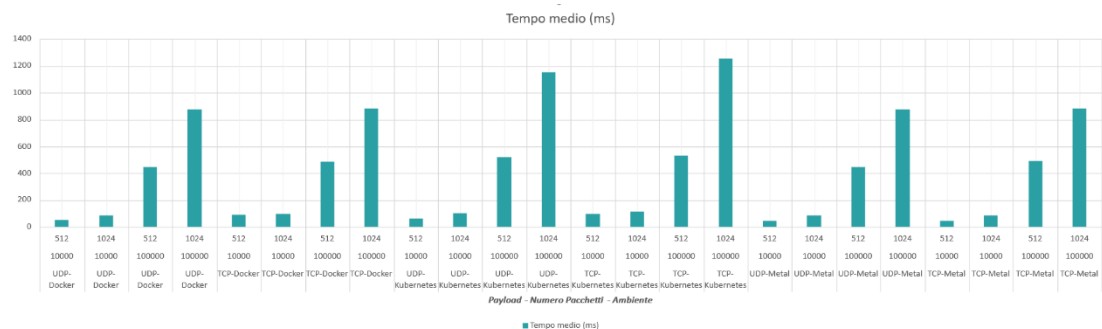
UDP/K8s			TCP/K8s		
Tempo (ms)	Payload	Numero	Tempo (ms)	Payload	Numero
65.100	512	10000	99.532	512	10000
64.864	512	10000	99.477	512	10000
65.202	512	10000	98.999	512	10000
64.703	512	10000	97.223	512	10000
64.659	512	10000	99.654	512	10000
100.149	1024	10000	116.338	1024	10000
99.883	1024	10000	115.233	1024	10000
101.339	1024	10000	116.899	1024	10000
98.322	1024	10000	116.411	1024	10000
100.272	1024	10000	116.565	1024	10000
521.333	512	100000	533.887	512	100000
521.566	512	100000	533.662	512	100000
522.992	512	100000	532.881	512	100000
521.302	512	100000	531.322	512	100000
522.185	512	100000	533.566	512	100000
1154.166	1024	100000	1252.110	1024	100000
1152.998	1024	100000	1254.034	1024	100000
1154.488	1024	100000	1253.185	1024	100000
1153.977	1024	100000	1253.481	1024	100000
1154.381	1024	100000	1252.234	1024	100000

4.5 Aggregazione dei dati

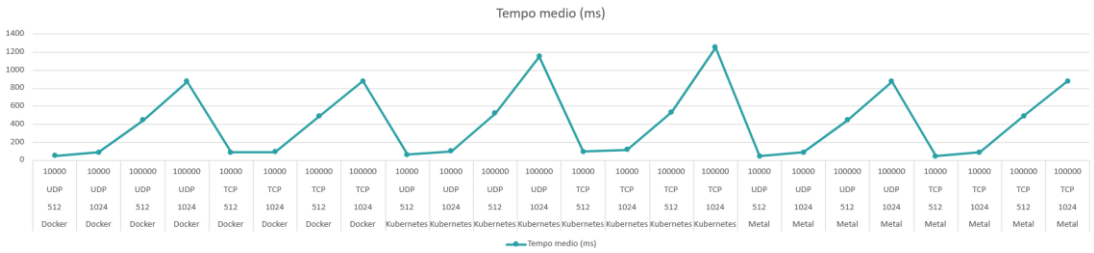
Scenario	Protocollo	Ambiente	Payload (B)	Numero Pacchetti	Tempo medio (ms)
UDP-Docker	UDP	Docker	512	10000	51,33
UDP-Docker	UDP	Docker	1024	10000	87,9
UDP-Docker	UDP	Docker	512	100000	447,56
UDP-Docker	UDP	Docker	1024	100000	875,56
TCP-Docker	TCP	Docker	512	10000	89,64
TCP-Docker	TCP	Docker	1024	10000	94,79
TCP-Docker	TCP	Docker	512	100000	488,82
TCP-Docker	TCP	Docker	1024	100000	881,26
UDP-Kubernetes	UDP	Kubernetes	512	10000	65
UDP-Kubernetes	UDP	Kubernetes	1024	10000	100
UDP-Kubernetes	UDP	Kubernetes	512	100000	522
UDP-Kubernetes	UDP	Kubernetes	1024	100000	1154
TCP-Kubernetes	TCP	Kubernetes	512	10000	98,98
TCP-Kubernetes	TCP	Kubernetes	1024	10000	116,3
TCP-Kubernetes	TCP	Kubernetes	512	100000	532,86
TCP-Kubernetes	TCP	Kubernetes	1024	100000	1253
UDP-Metal	UDP	Metal	512	10000	46,57
UDP-Metal	UDP	Metal	1024	10000	86,86
UDP-Metal	UDP	Metal	512	100000	447,08
UDP-Metal	UDP	Metal	1024	100000	874,98
TCP-Metal	TCP	Metal	512	10000	46,73
TCP-Metal	TCP	Metal	1024	10000	88
TCP-Metal	TCP	Metal	512	100000	491,07
TCP-Metal	TCP	Metal	1024	100000	880,88

4.6 Rappresentazione dei dati

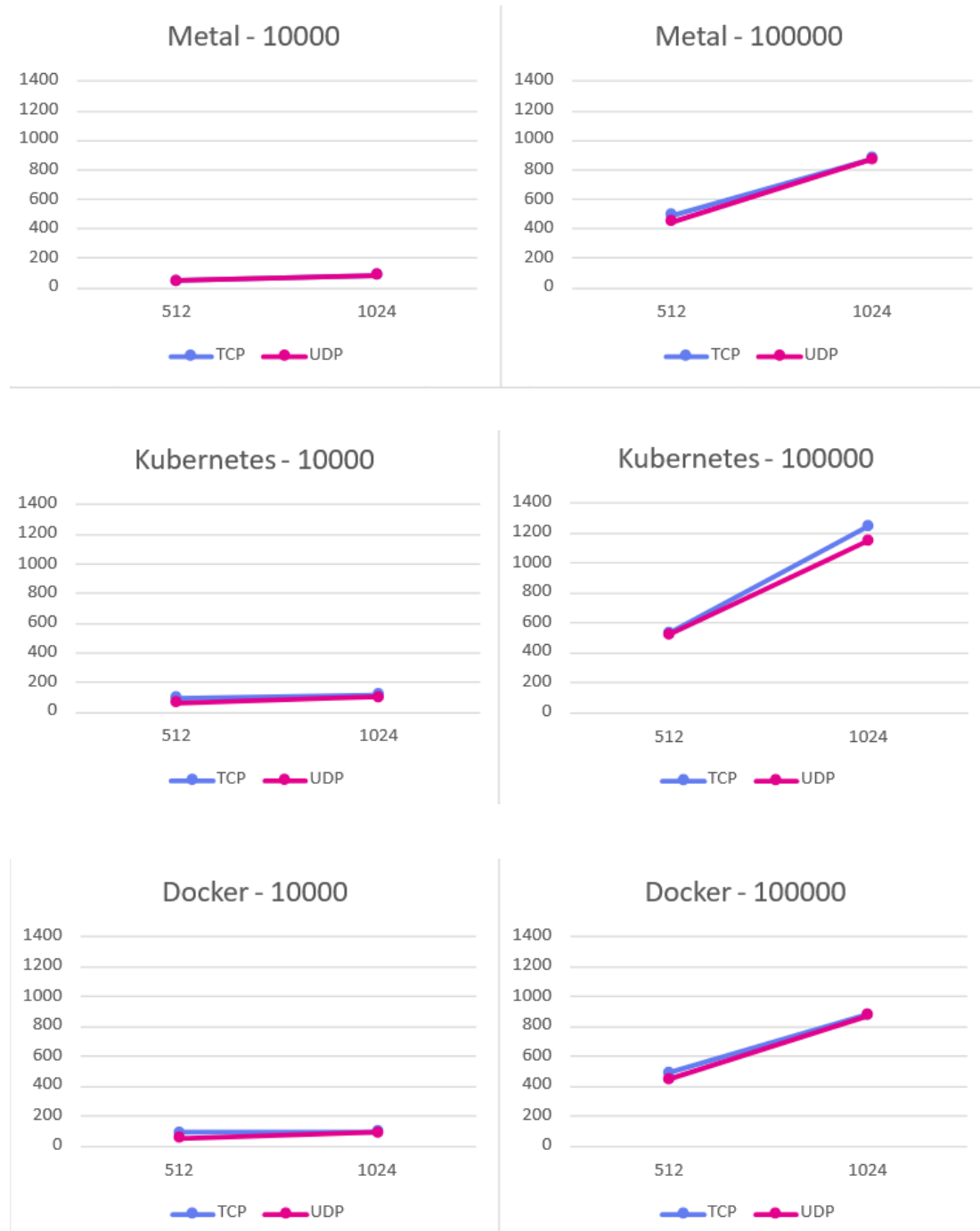
Per analizzare al meglio i dati ottenuti li ho rappresentati attraverso dei **grafici**. Il grafico a barre mostra al meglio l'**overhead** che Docker e Kubernetes aggiungono rispetto al bare metal. Si può notare come K8s introduca più latenza rispetto agli altri due scenari.



Il seguente grafico evidenzia come cresce il tempo rispetto al numero di pacchetti e alla grandezza del payload.



I grafici seguenti mostrano invece la **differenza di performance** nel caso si utilizzi il protocollo UDP o TCP nei vari ambienti. Sull'asse x è indicata la grandezza dei messaggi mentre sull'asse y il tempo totale della trasmissione in millisecondi.



4.7 Considerazioni

I test mostrano che i **tempi** di trasmissione utilizzando il protocollo UDP su Docker e Kubernetes **sono superiori** rispetto ai tempi ottenuti su bare metal probabilmente a causa dell'overhead dato dalla virtualizzazione. Il protocollo TCP risente maggiormente di questo aumento di latenza probabilmente a causa del controllo di flusso.

Per quanto riguarda la **scalabilità** del sistema possiamo vedere come il sistema scala in modo prevedibile all'aumentare del numero dei pacchetti. Non è quindi presente nessuno crollo improvviso delle prestazioni ma il costo **cresce linearmente** al carico.

Dai dati emerge anche la **differenza di performance tra Docker e Kubernetes**. L'overhead introdotto da Kubernetes, cioè la virtualizzazione e l'utilizzo di CNI, rende l'invio dei messaggi meno performante rispetto all'utilizzo di Docker. Quest'ultimo invece non ha una differenza molto marcata rispetto all'esecuzione di INSANE direttamente sulla macchina virtuale.